

# **LAPORAN PRAKTIKUM STRUKTUR DATA**

## **Implementasi Binary Tree dan Graph Traversal Menggunakan Java**



Disusun Oleh:

Shofwatul Hasanah Sarma

NIM 2511533015

Dosen Pengampu: Wahyudi. DR, S.T.M.T

**DEPARTEMEN INFORMATIKA**

**FAKULTAS TEKNOLOGI INFORMASI**

**UNIVERSITAS ANDALAS**

**PADANG**

**2026**

## KATA PENGANTAR

Puji dan syukur kita panjatkan kehadiran Allah yang Maha Esa karena telah memberikan nikmat sehat jasmani dan rohani sehingga saya bisa menyelesaikan laporan ini. Di dalam laporan ini, terdapat hasil praktikum, pembahasan serta kesimpulan yang saya susun secara sistematis. Saya sebagai penulis sekaligus penyusun laporan ini sangat berterimakasih kepada Bapak Wahyudi selaku dosen dalam mata kuliah praktikum Struktur Data Departemen Informatika, Universitas Andalas, yang telah menuntun dan mengajarkan saya mata kuliah ini.

Dalam penulisan laporan praktikum ini, saya mendapatkan suatu pelajaran baik serta mendapatkan pengetahuan tentang praktikum tersebut. Saya sebagai penulis dalam laporan ini masih dalam proses belajar, baik dalam tata penulisan laporan yang benar maupun dalam memahami penyajian laporan yang sesuai kaidah. Untuk itu saya berharap mendapatkan saran-saran yang membangun sehingga dapat meningkatkan pengetahuan saya dan dapat membuat isi laporan yang lebih baik, mendidik dan dapat dipertanggungjawabkan.

Padang, 2026

Shofwatul Hasanah Sarma

# DAFTAR ISI

|                                       |                                     |
|---------------------------------------|-------------------------------------|
| KATA PENGANTAR .....                  | i                                   |
| DAFTAR ISI.....                       | ii                                  |
| BAB I.....                            | 3                                   |
| PENDAHULUAN .....                     | <b>Error! Bookmark not defined.</b> |
| 1.1    Latar Belakang.....            | 3                                   |
| 1.2    Tujuan Praktikum.....          | 3                                   |
| 1.3    Manfaat Praktikum.....         | 4                                   |
| BAB II.....                           | 5                                   |
| PEMBAHASAN.....                       | 5                                   |
| 2.1    Pengertian .....               | 5                                   |
| 2.2    Praktikum .....                | 5                                   |
| BAB III .....                         | 49                                  |
| KESIMPULAN.....                       | 49                                  |
| 3.1    Ringkasan Hasil Praktikum..... | 49                                  |
| 3.2    Saran Pengembangan .....       | 50                                  |
| DAFTAR PUSTAKA .....                  | 51                                  |

# BAB I

## PENDAHULUAN

### 1.1 Latar Belakang

Struktur data merupakan bagian penting dalam pemrograman karena digunakan untuk menyimpan, mengatur, dan mengolah data secara efisien. Salah satu struktur data yang banyak dipelajari Adalah Binary Tree, yaitu struktur data pohon yang setiap simpulnya memiliki paling banyak dua anak, yaitu anak kiri dan anak kanan. Binary tree sering digunakan untuk memahami konsep hierarki, pencarian, serta penelusuran data.

Selain itu, struktur data lain yang tidak kalah penting Adalah Graph, yaitu Kumpulan simpul yang saling terhubung melalui sisi atau edge. Graph sangat berguna untuk merepresentasikan hubungan antar objek dalam dunia nyata, seperti jaringan komputer, peta, media social dan sistem navigasi.

Pada praktikum ini, peneliti atau mahasiswa mempelajari penerapan binary tree dan graph traversal menggunakan Bahasa pemrograman java. Melalui program yang dibuat, dapat dipahami bagaimana node saling terhubung, bagaimana data dihitung jumlah simpulnya, serta bagaimana penelusuran dilakukan menggunakan metode traversal seperti PreOrder, InOrder, PostOrder, DFS, dan BFS. Praktikum ini sangat penting untuk melatih pemahaman tentang algoritma dasar dan implementasi struktur data secara langsung dalam program.

### 1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar **Binary Tree** dan **Graph**.
2. Mempelajari cara membuat dan menghubungkan node pada Binary Tree.
3. Mempelajari cara menambahkan edge pada Graph tak berarah.

4. Memahami cara menghitung jumlah simpul pada Binary Tree.
5. Memahami proses traversal pada Binary Tree, yaitu PreOrder, InOrder, dan PostOrder.
6. Memahami proses traversal pada Graph, yaitu DFS dan BFS.
7. Melatih kemampuan implementasi struktur data menggunakan bahasa Java.
8. Mengetahui perbedaan hasil penelusuran pada Binary Tree dan Graph.

### 1.3 Manfaat Praktikum

Manfaat dari praktikum ini antara lain:

1. Menambah pemahaman tentang cara kerja struktur data Binary Tree dan Graph.
2. Melatih logika pemrograman dalam membangun hubungan antar node.
3. Membantu memahami konsep traversal secara rekursif dan iteratif.
4. Menjadi dasar untuk mempelajari struktur data yang lebih kompleks.
5. Melatih penggunaan class, object, method, dan rekursi dalam Java.
6. Memberikan gambaran nyata tentang bagaimana data diolah dalam bentuk hierarki dan jaringan.

# BAB II

## PEMBAHASAN

### 2.1 Pengertian

Implementasi Binary Tree dan Graph Traversal Menggunakan Java adalah praktikum yang membahas pembuatan serta pengolahan dua jenis struktur data, yaitu Binary Tree dan Graph, dengan menggunakan bahasa pemrograman Java.

Binary Tree digunakan untuk menyusun data secara hierarkis dalam bentuk pohon, sedangkan Graph digunakan untuk menunjukkan hubungan antar simpul. Traversal adalah proses menelusuri simpul-simpul tersebut dengan urutan tertentu. Pada Binary Tree digunakan traversal PreOrder, InOrder, dan PostOrder, sedangkan pada Graph digunakan DFS (Depth First Search) dan BFS (Breadth First Search).

Praktikum ini bertujuan agar pengguna memahami bagaimana struktur data dibentuk, dihubungkan, ditampilkan, dan ditelusuri menggunakan algoritma tertentu.

### 2.2 Praktikum

1. Class BtreeDriver\_2511533015  
Program utama yang digunakan untuk membuat, menggabungkan, dan menampilkan sebuah Binary Tree. Selain itu, program juga menghitung jumlah simpul (node) dan menampilkan isi pohon menggunakan beberapa metode traversal.

A. Kode program

```
package  
  
public class
```

```

public static void main(String[] args) {
    //Membuat Pohon
    Tree tree = new Tree();

    tree.out("Jumlah Simpul awal pohon: ")

    //Menambahkan simpul data 1
    tree.root = new Node(1);

    //Menjadikan simpul 1 sebagai root
    tree.root = tree.root;

    tree.out("Jumlah simpul jika hanya ada root")
    tree.out("Jumlah simpul: ")

    Node node2 = new Node(2);
    Node node3 = new Node(3);
    Node node4 = new Node(4);
    Node node5 = new Node(5);
    Node node6 = new Node(6);
    Node node7 = new Node(7);
    Node node8 = new Node(8);
    Node node9 = new Node(9);

    tree.root.left = node2;
    tree.root.right = node3;
    node2.left = node4;
    node2.right = node5;
    node3.left = node8;
    node3.right = node6;
    node6.left = node7;
    node6.right = node9;

    //set root
    tree.root = tree.root;

    tree.out("menampilkan simpul terakhir: ")
    tree.out("Jumlah simpul: ")
    tree.out("jumlah simpul: setelah simpul 7 ditambahkan")
    tree.out("InOrder: ")

    tree.out("\nPreorder: ")

    tree.out("\nPostorder: ")

    tree.out("\nmenampilkan simpul dalam bentuk pohon")
}

```

## B. Penjelasan

### 1. Deklarasi package

- ➔ `package pekan9_2511533015;`
  - berfungsi untuk menempatkan class ini ke dalam package bernama `pekan8_2511533015`

### 2. deklarasi class

- ➔ `public class BtreeDriver_2511533015 {`
  - membuat class utama yang akan menjalankan program binary tree

### 3. method main

- ➔ `public static void main(String[] args) {`
  - method `main ()` Adalah titik awal eksekusi program java

### 4. membuat objek binary tree

- ➔ `BTree_2511533015 tree = new BTree_2511533015();`
  - Kode ini membuat sebuah objek binary tree Bernama `tree`

Ilustrasi awal:

Kosong

### 5. Menghitung jumlah simpul awal

- ➔ `System.out.print("Jumlah Simpul awal pohon: ");`  
`System.out.println(tree.countNodes());`
  - Method `countNodes ()` digunakan untuk menghitung jumlah node dalam pohon

Karena pohon masih kosong, maka outputnya:

Jumlah Simpul awal pohon: 0

### 6. Membuat root

- ➔ `Node_2511533015 root = new Node_2511533015(1);`
    - Membuat node dengan nilai data 1
- 1



7. Menjadikan node 1 sebagai root

➔ `tree.setRoot(root);`

➤ node bernilai 1 dijadikan akar (root) dari binary tree

```
1
```

8. menghitung jumlah simpul setelah ada root

➔ `System.out.println("Jumlah simpul jika hanya ada root");`  
`System.out.println(tree.countNodes());`

➤ Karena hanya ada satu node, maka hasilnya:  
Jumlah simpul jika hanya ada root

```
1
```

9. Membuat node lain

➔ `Node_2511533015 node2 = new Node_2511533015(2);`  
`Node_2511533015 node3 = new Node_2511533015(3);`  
`Node_2511533015 node4 = new Node_2511533015(4);`  
`Node_2511533015 node5 = new Node_2511533015(5);`  
`Node_2511533015 node6 = new Node_2511533015(6);`  
`Node_2511533015 node7 = new Node_2511533015(7);`  
`Node_2511533015 node8 = new Node_2511533015(8);`  
`Node_2511533015 node9 = new Node_2511533015(9);`

➤ Membuat node dengan data 2 sampai 9

10. Menghubungkan antar node

➔ Root ke node 2

➤ `root.setLeft(node2);`

```
1
/
2
```

➔ node 2 ke node 4

➤ `node2.setLeft(node4);`

```
1
/
2
/
4
```

➔ node 2 ke node 5

➤ node2.setRight(node5);

```
1
 /
2
 /\
4 5
```

➔ node 2 ke node 8

➤ node2.setRight(node8);

perlu diperhatikan!

Baris ini menggantikan node 5

Awalnya:

```
2
 /\
4 5
```

Menjadi:

```
2
 /\
4 8
```

Node 5 tidak lagi terhubung dengan pohon

➔ root ke node 3

➤ root.setRight(node3);

```
1
 /\
2 3
 /\
4 8
```

➔ node 3 ke node 6

➤ node3.setLeft(node6);

```
1
 /\
2 3
 /\ /
4 8 6
```

➔ node 3 ke node 7

➤ node3.setRight(node7);

```
1
 / \
2   3
 / \ / \
4 8 6 7
```

➔ node 6 ke node 9

➤ node6.setLeft(node9);

struktur akhir pohon:

```
1
 / \
2   3
 / \ / \
4 8 6 7
 /
9
```

Catatan:

Node 5 dibuat tetapi tidak masuk ke pohon karena telah diganti oleh node b

11. menentukan current node

➔ tree.setCurrent(tree.getRoot());

➤ menjadikan root sebagai node aktif (current)

➔ Current = 1

12. Menampilkan data current

➔ System.out.println(tree.getCurrent().getData());

Output

➔ 1

13. Menghitung jumlah simpul

➔ System.out.println(tree.countNodes());

➤ Node yang benar benar terhubung:

- 1
- 2
- 3
- 4

- 5
- 6
- 7
- 8
- 9

Total:

➔ 8 simpul

➤ Node 5 tidak dihitung karena tidak terhubung

#### 14. Traversal inOrder

➔ `tree.printInorder();`

➤ urutan:

kiri -> root -> kanan

proses:

- 4
- 2
- 8
- 1
- 9
- 6
- 3
- 7

Hasil:

➤ 4 2 8 1 9 6 3 7

#### 15. Traversal preorder

➔ `tree.printPreOrder();`

➤ urutan:

root – kanan – kiri

hasil:

1 2 4 8 3 6 9 7

#### 16. traversal postorder

➔ `tree.printPostOrder();`

➤ urutan:

kiri – kanan – root

hasil:

4 8 2 9 6 7 3 1

17. menampilkan bentuk pohon

➔ tree.print();

- method ini biasanya digunakan untuk mencetak struktur binary tree dalam bentuk visual.

Contoh tampilan yang mungkin:

```
      1
     / \
    2   3
   /\  /\
  4 8 6 7
   /
  9
```

C. Output

```
2C <terminated> BtreeDriver_2511533015 [Java Application]
Pr |Jumlah Simpul awal pohon: 0
   |Jumlah simpul jika hanya ada root
   |1
   |menampilkan simpul terakhir:
   |1
   |jumlah simpul: setelah simpul 7 ditambahkan
   |8
   |InOrder:
   |4 2 5 1 9 6 3 7
   |Preorder:
   |1 2 4 5 3 6 9 7
   |Postorder:
   |4 5 2 9 6 7 3 1
   |menampilkan simpul dalam bentuk pohon
   || /--7
   || /--3
   || \--6
   || \--9
   || \--1
   || /--5
   || \--2
   || \--4
```

2. Kelas GraphTraversal\_2511533015

Program ini digunakan untuk membuat graf tak berarah dan melakukan penelusuran graf menggunakan algoritma DFS (Depth First Search) dan BFS (Breadth First Search).

Graf disimpan menggunakan struktur data dan adjacency list dengan bantuan HashMap

A. Kode program

```
package  
  
import  
  
public class  
  
    private  
        graph new  
  
    // Menambahkan edge (graf tak terarah)  
    public void  
        node1 node2  
        graph node1 new  
        graph node2 new  
  
        graph node1 node2  
        graph node2 node1  
  
    // Menampilkan graf  
    public void  
        out "Graf Awal (Adjacency List): "  
  
        for node graph  
            out node "-> "  
                neighbors graph node  
            out ", " neighbors  
  
        out  
  
    // DFS Rekursif
```

```

public void DFSIteratif(int start) {
    boolean[] visited = new boolean[n];

    out.println("Penelusuran DFS: ");
    DFSIteratif(start, visited);
}

private void DFSIteratif(int current, boolean[] visited) {
    if (visited[current])
        return;

    visited[current] = true;
    out.println(current + " ");

    for (int neighbor : graph.get(current))
        if (!visited[neighbor])
            DFSIteratif(neighbor, visited);
}

// BFS Iteratif
public void BFSIteratif(int start) {
    boolean[] visited = new boolean[n];
    Queue<Integer> queue = new Queue<>();

    queue.add(start);
    visited[start] = true;

    out.println("Penelusuran BFS: ");

    while (!queue.isEmpty()) {
        int current = queue.poll();
        out.println(current + " ");

        for (int neighbor : graph.get(current))
            if (!visited[neighbor])
                queue.add(neighbor);
    }
}

```

```

        queue    neighbor

    }

    out

// Main
public static void main(String[] args) {
    Graph g = new Graph(4);
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 3);

    // Cetak graf
    out.println("Graf awal adalah:");
    g.print();

    // Traversal
    g.bfs(0);
    g.dfs(0);
}

```

## B. Penjelasan

### 1. Deklarasi package

➔ package pekan9\_2511533015;

- kode ini menunjukkan bahwa class berada di dalam package pekan9\_2511533015

### 2. import library

➔ import java.util.\*;

- digunakan untuk mengimpor seluruh class dari package java.util, seperti:



- map
- hashmap
- list
- arraylist
- set
- hashset
- queue
- linkedlist

semua struktur data tersebut digunakan dalam pembuatan graf dan proses traversal

### 3. deklarasi class

➔ `public class GraphTraversal_2511533015 {`

➤ class ini berfungsi untuk:

- membuat graf
- menambahkan edge
- menampilkan graf
- menjalankan DFS
- menjalankan BFS

### 4. membuat struktur graf

➔ `private Map<String, List<String>> graph = new HashMap<>();`

➤ variable graph digunakan untuk menyimpan hubungan antar simpul

bentuk penyimpanannya:

➔ Node -> Daftar Tetangga

➤ Misalnyaa:

A -> B, C

B -> A, D, C

C -> A, B

D -> B

➤ Disimpan seperti:

{

A=[B,C],

B=[A,D,C],

```
C=[A,B],
D=[B]
}
```

#### 5. Method addEdge ()

➔ `public void addEdge(String node1, String node2) {`  
 ➤ method ini digunakan untuk menambahkan hubungan (edge) antara dua simpul

# membuat node jika belum ada:

➔ `graph.putIfAbsent(node1, new ArrayList<>());`  
`graph.putIfAbsent(node2, new ArrayList<>());`  
 ➤ artinya:  
 jika node belum ada di dalam graf, maka node baru dengan daftar tetangga kosong.

Misalnya:

Sebelum:

```
{}
```

Setelah:

```
A -> []
```

```
B -> []
```

# menambahkan hubungan antar node

➔ `graph.get(node1).add(node2);`  
`graph.get(node2).add(node1);`  
 ➤ karena graf bersifat tak berarah (undirected graph), maka hubungan dibuat dua arah

misalnya:

```
graph.addEdge("A","B");
```

hasil:

```
A -> B
```

```
B -> A
```

#### 6. method printGraph()

➔ `public void printGraph() {`

- method ini digunakan untuk menampilkan isi graf dalam bentuk adjacency list

# menampilkan judul

➔ `System.out.println("Graf Awal (Adjacency List): ");`

# melakukan perulangan semua node

➔ `for (String node : graph.keySet())`

- `graph.keySet()` mengambil semua simpul yang ada pada graf.

Misalnya:

A

B

C

D

# mengambil daftar tetangga

➔ `List<String> neighbors = graph.get(node);`

Contoh:

Jika node = A

Maka:

`neighbors = [B,C]`

# menampilkan tetangga

➔ `System.out.println(String.join(", ", neighbors));`

Output:

A -> B, C

B -> A, D, C

C -> A, B

D -> B

## 7. Method DFS (Depth First Search)

➔ `public void dfs(String start)`

- DFS Adalah algoritma penelusuran graf yang bekerja dengan cara:

Mengunjungi satu jalur sedalam mungkin, kemudian Kembali ke simpul sebelumnya.

# membuat set visited

- ➔ `Set<String> visited = new HashSet<>();`
  - Digunakan untuk menyimpan node yang sudah dikunjungi.
- Tujuannya agar tidak terjadi perulangan tak terbatas.

# memanggil method rekursif

- ➔ `dfsHelper(start, visited);`
  - jika dipanggil:  
`graph.dfs("A");`  
maka proses dimulai dari node A

#### 8. method `dfsHelper()`

- ➔ `private void dfsHelper(String current, Set<String> visited)`
  - method ini menjalankan DFS secara rekursif

# mengecek apakah node sudah dikunjungi

- ➔ `if (visited.contains(current))`
  - `return;`
  - jika node sudah pernah dikunjungi, maka fungsi berhenti

# menandai node telah dikunjungi

- ➔ `visited.add(current);`
  - misalnya:  
`Visited = {A}`

# menampilkan node

- ➔ `System.out.print(current + " ");`
  - Output sementara:  
A

# mengunjungi semua tetangga

- ➔ for (String neighbor : graph.getOrDefault(current, new ArrayList<>()))
  - mengambil seluruh tetangga dari node saat ini kemudian:
- ➔ dfsHelper(neighbor, visited);
  - DFS akan terus masuk ke node berikutnya sampai tidak ada jalur lagi.

# proses DFS pada Graf

Graf:

```

      A
     /\
    B---C
    |
    D
  
```

Urutan DFS:

```

A
↓
B
↓
D
↑
B
↓
C
  
```

Output:

A B D C

## 9. Method BFS (Breadth First Search)

- ➔ public void bfs(String start)
  - BFS Adalah algoritma penelusuran graf yang bekerja dengan cara:  
Mengunjungi semua tetangga terlebih dahulu sebelum masuk ke Tingkat berikutnya:  
BFS menggunakan struktur data Queue

# membuat set visited

- ➔ Set<String> visited = new HashSet<>();
  - Menyimpan node yang sudah dikunjungi

# membuat queue

- ➔ Queue<String> queue = new LinkedList<>();
  - Queue bekerja dengan prinsip:  
FIFO (First In First Out)

# menambahkan node awal

- ➔ queue.add(start);  
visited.add(start);
  - jika start = A  
maka:  
Queue = [A]  
Visited = {A}

# selama queue tidak kosong

- ➔ while (!queue.isEmpty())
  - BFS akan terus berjalan

# mengambil node paling depan

- ➔ String current = queue.poll();
  - Misalnya:  
Queue = [A]
  - Setelah pol:  
Current = A  
Queue = []

# menampilkan node

- ➔ System.out.print(current + " ");
  - Output:  
A

```
# mengecek semua tetangga
➔ for (String neighbor : graph.getDefault(current, new
    ArrayList<>()))
    ➤ jika tetangga belum dikunjungi:
➔ visited.add(neighbor);
  queue.add(neighbor);
misalnya:
Queue = [B,C]
```

# proses BFS

Graf:

```

  A
 /\
B---C
 |
  D
```

Langkah:

Queue=[A]

A keluar

Masuk B,C

Queue=[B,C]

B keluar

Masuk D

Queue=[C,D]

C keluar

Queue=[D]

D keluar

Output:

A B C D

#### 10. method main

- ➔ `public static void main(String[] args)`
  - merupakan tempat program dijalankan

# membuat objek graf

- ➔ `GraphTraversal_2511533015 graph =  
new GraphTraversal_2511533015();`

# menambahkan edge

- ➔ `graph.addEdge("A", "B");  
graph.addEdge("A", "C");  
graph.addEdge("B", "D");  
graph.addEdge("B", "C");`
  - graf yang terbentuk:

```
A
 /\
B---C
 |
D
```

# menampilkan graf

- ➔ `graph.printGraph();`
  - output yang mungkin:  
Graf Awal (Adjacency List):

```
A -> B, C
B -> A, D, C
C -> A, B
D -> B
```



(urutan dapat berubah karena hashmap tidak menjamin urutan data)

# menjalankan DFS

➔ graph.dfs("A");

➤ output:

Penelusuran DFS:

A B D C

# menjalankan BFS

➔ graph.bfs("A");

➤ output:

Penelusuran BFS:

A B C D

Perbedaan DFS dan BFS

DFS:

- menggunakan rekursi atau stack
- menelusuri sedalam mungkin
- jalur A – B – D – C
- lebih hemat memori pada graf tertentu

BFS:

- menggunakan queue
- menelusuri per level
- jalur: A – B – C -D
- cocok mencari jalur terpendek pada graf tak berbobot

C. Output

```
<terminated> GraphTraversal_2511533015 [Java Application] C:\Users\user\
Graf awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, C
C -> A, B
D -> B

Penelusuran DFS:
A B D C
Penelusuran BFS:
A B C D
```

### 3. Kelas node\_2511533015

Program ini merupakan class yang digunakan untuk membentuk simpul (node) pada binary tree.

Setiap node memiliki:

^ data (nilai yang disimpan)

^ anak kiri (left child)

^ anak kanan (right child)

Selain itu, class ini juga memiliki method untuk:

^ menambahkan anak kiri dan kanan

^ mengambil data node

^ mengubah data node

^ melakukan traversal (PreOrder, InOrder, PostOrder)

^ menampilkan struktur pohon

#### A. Kode program

```
package
public class
```

```

int data
    left
    right

public      int data
    this data = data
    left = null
    right = null

public void      node
    if left == null
        left = node

public void      node
    if right == null
        right = node

public
    return left

public
    return right

public int
    return data;

public void      int data
    this data = data

void      node
    if node == null
        return
    out.println("node data: " +
        node.left
        node.right)

void      node
    if node == null
        return
    node.left
    node.right
    out.println("node data: ")

```

```

void printInOrder(Node node) {
    if (node == null)
        return;
    printInOrder(node.left);
    out.print(node.data + " ");
    printInOrder(node.right);
}

public boolean isTail() {
    return this.isTail;
}

public boolean isTail(String sb) {
    if (right == null)
        return prefix + isTail + "|" + " " + false + sb;

    out.print(prefix + isTail + "\\--" + "/--" + data + " ");
    if (left == null)
        return left + prefix + isTail + " " + "|" + true + sb;

    return sb;
}

```

## B. Penjelasan

### 1. Deklarasi package

➔ package pekan9\_2511533015;

➤ kode ini menunjukkan bahwa classs berada pada package pekan

### 2. deklarasi class

➔ public class Node\_2511533015 {

➤ class ini digunakan untuk merepresentasikan satu simpul dalam binary tree

### 3. deklarasi variable

➔ int data;

➔ Node\_2511533015 left;

➔ Node\_2511533015 right;

➤ Variable variabel tersebut memiliki fungsi:

- Data : menyimpan nilai node
- Left : menyimpan Alamat anak kiri
- Right : menyimpan Alamat anak kanan

Misalnya dibuat:

```
Node_2511533015 root = new Node_2511533015(10);
```

10

/ \

null null

#### 4. Constructor

```
➔ public Node_2511533015(int data) {
    this.data = data;
    left = null;
    right = null;
}
```

➤ contruktor digunakan Ketika objek node dibuat

contoh:

```
Node_2511533015 node1 = new Node_2511533015(1);
```

Proses yang terjadi:

- data = 1
- left = null
- right = null

ilustrasi:

1

/ \

null null

#### 5. method setLeft ()

```
➔ public void setLeft(Node_2511533015 node) {
```

```

if (left == null)
    left = node;
}

```

➤ method ini digunakan untuk menghubungkan anak kiri  
 contoh:  
 root.setLeft(node2);

sebelum:

```

    1
   /
  Null

```

Sesudah:

```

    1
   /
  2

```

#### 6. method setRight()

```

➔ public void setRight(Node_2511533015 node) {
    if (right == null)
        right = node;
}

```

➤ method ini digunakan untuk menghubungkan anak kanan  
 contoh:  
 root.setRight(node3);

hasil:

```

    1
   /\
  2 3

```

Sama seperti setLeft(), method ini hanya mengisi anak kanan  
 jika masih kosong.

#### 7. Method getLeft()

```

➔ public Node_2511533015 getLeft() {
    return left;
}

```

```
}
```

➤ digunakan untuk mengambil node anak kiri

contoh:

```
Node_2511533015 x = root.getLeft();
```

Jika:

1

/

2

Maka:

x = node 2

8. method `getRight()`

```
➔ public Node_2511533015 getRight() {  
    return right;  
}
```

➤ digunakan untuk mengambil node anak kanan

9. method `getData()`

```
➔ public int getData() {  
    return data;  
}
```

➤ digunakan untuk mengambil nilai yang tersimpan pada node

contoh:

```
System.out.println(root.getData());
```

Output:

1

10. method `setData()`

```
➔ public void setData(int data) {  
    this.data = data;  
}
```

➤ digunakan untuk mengubah nilai data pada node

contoh:

```
root.setData(100);
```

sebelum:

1

Sesudah:

100

#### 11. method printPreorder()

```
➔ void printPreorder(Node_2511533015 node) {
```

```
    if (node == null)
```

```
        return;
```

```
    System.out.print(node.data + " ");
```

```
    printPreorder(node.left);
```

```
    printPreorder(node.right);
```

```
}
```

➤ method ini melakukan traversal preorder

urutan:

Root → Left → Right

Misalnya pohon:

1

/ \

2 3

/ \

4 5

Proses:

1

2

4

5

3

Output:

1 2 4 5 3

#### 12. method printPostorder()

```
➔ void printPostorder(Node_2511533015 node) {
```

```
    if (node == null)
```



```

        return;

        printPostorder(node.left);
        printPostorder(node.right);
        System.out.print(node.data + " ");
    }

```

➤ traversal PostOrder memiliki urutan:

Left → Right → Root

Pohon:

```

    1
   /\
  2 3
 /\
4  5

```

Proses:

```

4
5
2
3
1

```

### 13. method printInorder

```

➔ void printInorder(Node_2511533015 node) {
    if (node == null)
        return;

```

```

        printInorder(node.left);
        System.out.print(node.data + " ");
        printInorder(node.right);
    }

```

➤ traversal InOrder memiliki urutan:

Left → Root → Right

Contoh:

```

1

```

```

    /\
   2 3
  /\
 4 5

```

Proses:

```

4
2
5
1
3

```

Output:

```

4 2 5 1 3

```

#### 14. method print()

```

➔ public String print() {
    return this.print("", true, "");
}

```

- method ini digunakan untuk menampilkan binary tree dalam bentuk visual

method ini memanggil method berikut:

```

public String print(String prefix, boolean isTail, String sb)

```

#### 15. method print (string prefix, Boolean isTail, String sb)

```

➔ public String print(String prefix, boolean isTail, String sb) {
    ➤ method ini bekerja secara rekursif untuk mencetak bentuk pohon

```

```

    # menampilkan anak kanan terlebih dahulu

```

```

    if (right != null) {
        right.print(prefix + (isTail ? "| " : " "), false, sb);
    }

```

```

    # menampilkan node saat ini

```

```

    System.out.println(prefix + (isTail ? " \n" : "/--") + data);

```

```
# menampilkan anak kiri
if (left != null) {
    left.print(prefix + (isTail ? " " : "| "), true, sb);
}
```

# misalnya pohon:

```
  1
 / \
2   3
 / \
4   5
```

# tampilan yang dihasilkan kira kira:

```
|/--3
 \--1
 |/--5
 \--2
 \--4
```

Tujuan agar struktur pohon lebih mudah dipahamai

- cara kerja keseluruhan class

misalnya program:

```
Node_2511533015 root = new Node_2511533015(1);
Node_2511533015 node2 = new Node_2511533015(2);
Node_2511533015 node3 = new Node_2511533015(3);
```

```
root.setLeft(node2);
root.setRight(node3);
```

maka binary tree yang terbentuk:

```
  1
 / \
2   3
```

Jika dipanggil:

```
root.printPreorder(root);
```

output:

1 2 3

Jika dipanggil:

```
root.printInorder(root);
```

output:

2 1 3

Jika dipanggil:

```
root.printPostorder(root);
```

output:

2 3 1

- Kesimpulan

Class `node_2511533015` adalah class dasar yang digunakan untuk membangun binary tree. Setiap objek menyimpan sebuah data serta referensi ke anak kiri dan anak kanan.

Class ini juga menyediakan operasi penting seperti:

1. Menambah anak kiri dan kanan
2. Mengambil dan mengubah data
3. Traversal PreOrder
4. Traversal InOrder
5. Traversal PostOrder
6. Menampilkan struktur binary tree

Class ini menjadi fondasi utama bagi class

`BTree_2511533015` dan `BTreeDriver_2511533015`

dalam menjalankan operasi pada pohon biner.

### C. Output

[illegible]

4. Kelas BTree\_2511533015

### A. Kode program

```
package com.leetcode;

public class Solution {
    private static Node root;
    private static Node currentNode_2511533015;

    public static Node root = null;

    public static boolean insert(int data) {
        return root.insert(data);
    }

    private boolean insert(Node node, int data) {
        if (node == null) {
            return true;
        }
    }
}
```

```

        if (node == null)
            if (node == data)
                return true;

            if (node == null)
                if (node == data)
                    return true;

            return false;

    public void
        root.printlnorder root

    public void
        root root

    public void
        root root

    public
        return root

    public boolean
        return root == null

    public int
        return root

    private int node
        int count
        if (node == null)
            return
        else
            count = node
            count = node
            return count

    public void
        root

    public
        return currentNode_2511533015

```

```

public void                node
    this currentNode_2511533015 node

public void                root
    this root root

```

## B. Penjelasan

### 1. Deklarasi package

- ➔ package pekan9\_2511533015;
- class berada di dalam package pekan9\_2511533015

### 2. deklarasi class

- ➔ public class BTree\_2511533015 {
- class ini digunakan sebagai pengelola binary tree

### 3. deklarasi variabel

- ➔ private Node\_2511533015 root;
- private Node\_2511533015 currentNode\_2511533015;
- # root

- ➔ private Node\_2511533015 root;
- menyimpan simpul paling atas (akar pohon)

contoh:

```

    1
   /\
  2 3

```

Root:

1

# current node

- ➔ private Node\_2511533015 currentNode\_2511533015;
- digunakan untuk menyimpan node yang sedang aktif atau sedang diproses
- misalnya:

```
tree.setCurrent(tree.getRoot());
```

maka current node Adalah:

1

#### 4. constructor

```
➔ public BTree_2511533015() {  
    root = null;  
}
```

➤ Ketika objek binary tree dibuat, pohon masih kosong

Contoh:

```
BTree_2511533015 tree = new BTree_2511533015();
```

Keadaan awal:

Kosong

#### 5. Method search()

```
➔ public boolean search(int data) {  
    return search(root, data);  
}
```

➤ method ini digunakan untuk mencari data pada binary tree  
method ini memanggil method rekursif:

```
➔ private boolean search(Node_2511533015 node, int data)
```

#### 6. method search (node, data)

```
➔ private boolean search(Node_2511533015 node, int data)
```

➤ method ini mencari data secara rekursif

```
# mengecek data node
```

```
if (node.getData() == data)  
    return true;
```

➤ jika data ditemukan, maka mengembalikan nilai:

```
true
```

```
# mencari pada anak kiri
```

```
if (node.getLeft() != null)  
    if (search(node.getLeft(), data))
```



```
return true;
```

- jika data belum ditemukan, program akan mencari ke subtree kiri

```
# mencari pada anak kanan
```

```
if (node.getRight() != null)
```

```
    if (search(node.getRight(), data))
```

```
        return true;
```

- jika belum ditemukan juga, program mencari ke subtree kanan

```
# jika data tidak ditemukan
```

```
return false;
```

contoh pohon:

```
    1
   /\
  2 3
 /\
4  5
```

Jika:

```
tree.search(5);
```

proses:

```
1
↓
2
↓
4
↑
2
↓
5
```

Hasil:

True

7. method printInorder()

```
➔ public void printInorder() {  
    root.printInorder(root);  
}
```

➤ menampilkan isi binary tree menggunakan traversal InOrder

urutan:

Left → Root → Right

Contoh:

```
    1  
  /\   
 2 3  
 /\   
4 5
```

Output:

4 2 5 1 3

8. method printPreOrder()

```
➔ public void printPreOrder() {  
    root.printPreorder(root);  
}
```

➤ traversal PreOrder

urutan:

Root → Left → Right

Output:

1 2 4 5 3

9. method printPostOrder()

```
➔ public void printPostOrder() {  
    root.printPostorder(root);  
}
```

➤ traversal PostOrder

urutan:

Left → Right → Root

Output:

4 5 2 3 1

10. method `getRoot()`

```
➔ public Node_2511533015 getRoot() {  
    return root;  
}
```

➤ digunakan untuk mengambil node root

contoh:

Node\_2511533015 x = tree.getRoot();

Jika root bernilai 1:

x = 1

11. method `isEmpty()`

```
➔ public boolean isEmpty() {  
    return root == null;  
}
```

➤ digunakan untuk mengecek apakah binary tree kosong

jika:

root = null

maka:

true

jika:

root = 1

maka:

false

12. method `countNodes()`

```
➔ public int countNodes() {  
    return countNodes(root);  
}
```

}

- digunakan untuk menghitung jumlah simpul dalam binary tree  
method ini memanggil method rekursif  
private int countNodes(Node\_2511533015 node)

13. method countNodes(Node)

➔ private int countNodes(Node\_2511533015 node)

- method ini menghitung jumlah seluruh node secara rekursif.

# nilai awal

int count = 1;

^ setiap node dihitung Satu.

# jika node kosong

if (node == null) {

return 0;

}

^ tidak ada node yang dihitung.

# menghitung subtree kiri

count += countNodes(node.getLeft());

# menghitung subtree kanan

count += countNodes(node.getRight());

# misalnya pohon:

1

/ \

2 3

/ \

4 5

Perhitungan:

Node 4 = 1

Node 5 = 1

Node 2 = 1+1+1 = 3

Node 3 = 1

Node 1 = 1+3+1 = 5

Output:

5

14. method print()

```
➔ public void print() {  
    root.print();  
}
```

➤ digunakan untuk menampilkan binary tree dalam bentuk visual

contoh:

```
    1  
   /\n  2 3  
 /\n4  5
```

15. method getCurrent()

```
➔ public Node_2511533015 getCurrent() {  
    return currentNode_2511533015;  
}
```

➤ mengambil node yang sedang aktif

16. method setCurrent()

```
➔ public void setCurrent(Node_2511533015 node) {  
    this.currentNode_2511533015 = node;  
}
```

➤ mengatur node yang sedang aktif

contoh:

```
tree.setCurrent(tree.getRoot());
```

current node menjadi:

1

17. method setRoot()

```
public void setRoot(Node_2511533015 root) {
```

```
this.root = root;  
}
```

- digunakan untuk menentukan akar binary tree

contoh:

```
Node_2511533015 root = new Node_2511533015(1);  
tree.setRoot(root);
```

hasil:

1

Menjadi root pohon

- cara kerja program

misalnya:

```
Node_2511533015 root = new Node_2511533015(1);  
Node_2511533015 node2 = new Node_2511533015(2);  
Node_2511533015 node3 = new Node_2511533015(3);
```

```
root.setLeft(node2);  
root.setRight(node3);
```

```
tree.setRoot(root);
```

pohon yang terbentuk:

```
  1  
 /\n2 3
```

Jika dijalankan:

```
tree.countNodes();
```

output:

3

Jika:

```
tree.search(2);
```

output:

true

jika:

```
tree.printInorder();
```

output:

2 1 3

Jika:

```
tree.printPreOrder();
```

output:

1 2 3

Jika:

```
tree.printPostOrder();
```

output:

2 3 1

- hubungan antar class

BtreeDriver\_2511533015

|

|

v

BTree\_2511533015

|

|

v

Node\_2511533015

- node\_2511533015 = membuat simpul pohon
- BTree\_2511533015 = mengelola seluruh operasi Binary Tree
- BtreeDriver\_2511533015 = menjalankan program dan membangun pohon

- Kesimpulan

Class BTree\_2511533015 berfungsi sebagai pengelola utama binary tree yang menyediakan berbagai operasi penting, yaitu:

1. Menentukan root pohon
2. Mencari data dalam pohon (search)
3. Menghitung jumlah simpul (countNodes)
4. Mengecek apakah pohon kosong (isEmpty)
5. Menampilkan traversal:
  - ^ InOrder
  - ^ PreOrder
  - ^ PostOrder
6. Menampilkan bentuk visual pohon
7. Mengatur dan mengambil current node

Class ini menjadi inti dari implementasi binary tree dan bekerja Bersama class node\_2511533015 serta BtreeDriver\_2511533015 untuk membangun dan mengelola struktur pohon biner.

### C. Output





# BAB III

## KESIMPULAN

### 3.1 Ringkasan Hasil Praktikum

#### 1. Hasil pada Binary Tree

Pada class BtreeDriver\_2511533015, program berhasil membentuk Binary Tree dengan root bernilai 1. Node-node lain dihubungkan ke kiri dan kanan sehingga terbentuk struktur pohon. Dari hasil program, diketahui bahwa:

- Jumlah simpul awal pohon adalah 0.
- Setelah root dibuat, jumlah simpul menjadi 1.
- Setelah node-node lain dihubungkan, jumlah simpul yang benar-benar terhubung adalah 8, karena node 5 tertimpa oleh node 8 pada bagian node2.setRight(node8).
- Current node berhasil diarahkan ke root.
- Traversal menghasilkan urutan:
  - InOrder: 4 2 8 1 9 6 3 7
  - PreOrder: 1 2 4 8 3 6 9 7
  - PostOrder: 4 8 2 9 6 7 3 1
- Program juga dapat menampilkan bentuk pohon secara visual.

Hasil ini menunjukkan bahwa Binary Tree berhasil dibentuk dan traversal berjalan sesuai urutan metode masing-masing.

#### 2. Hasil pada Graph Traversal

Pada class `GraphTraversal_2511533015`, program berhasil membentuk graf tak berarah dengan simpul A, B, C, dan D. Hubungan antar simpul dibentuk menggunakan edge sehingga adjacency list terbentuk dengan baik.

Dari hasil program:

- Graf berhasil ditampilkan dalam bentuk adjacency list.
- DFS dari node A menghasilkan urutan penelusuran yang menelusuri simpul secara mendalam terlebih dahulu.
- BFS dari node A menghasilkan urutan penelusuran berdasarkan tingkat kedekatan simpul.
- DFS dan BFS sama-sama berhasil dijalankan tanpa error.

Hasil ini membuktikan bahwa program graf sudah mampu menyimpan hubungan antar simpul dan melakukan traversal dengan benar.

## 3.2 Saran Pengembangan

Beberapa saran untuk pengembangan praktikum ini adalah:

1. Menambahkan pengecekan error pada saat root masih kosong agar program tidak menimbulkan `NullPointerException`.
2. Memperbaiki pengisian node agar tidak ada node yang tertimpa, seperti kasus node 5 yang digantikan oleh node 8.
3. Menambahkan fitur pencarian node pada Graph, misalnya pencarian jalur terpendek.
4. Menambahkan operasi penghapusan node atau edge pada Graph maupun Binary Tree.
5. Membuat tampilan output yang lebih rapi dan mudah dibaca.

6. Mengembangkan program agar mampu menyimpan data bertipe selain integer dan string.
7. Menambahkan komentar kode yang lebih lengkap agar mudah dipahami saat dibaca kembali.
8. Menguji program dengan data yang lebih banyak agar kemampuan traversal terlihat lebih jelas.

## DAFTAR PUSTAKA

1. Abdul Kadir. Algoritma dan Pemrograman Menggunakan Java. Yogyakarta: Andi.
2. Kurniawan, A. Pemrograman Struktur Data dengan Java. Jakarta: Elex Media Komputindo.
3. Sutrisno, H. Struktur Data dan Algoritma. Bandung: Informatika.
4. Oracle. Java Documentation. Sun/Oracle Java API Documentation.
5. Prasetyo, D. Konsep Dasar Struktur Data: Stack, Queue, Tree, dan Graph. Yogyakarta: Graha Ilmu.

# TUGAS

## 1. Kode program

```
package tugaspekan9_2511533015;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.*;

// Kelas utama dengan akhiran NIM 2511533015
public class PencarianJalur_2511533015 extends JFrame {
    // Variabel dengan akhiran _3015
    private Graph_2511533015 graph_3015;
    private JComboBox<String> startCombo_3015;
    private JComboBox<String> goalCombo_3015;
    private JTextArea resultArea_3015;
    private DrawingPanel_2511533015 drawingPanel_3015;
    private Set<String> visitedNodesSet_3015;
    private Set<String> pathNodesSet_3015;
    private List<String> lastVisitedOrder_3015;
    private int lastExploredCount_3015;

    public PencarianJalur_2511533015() {
        // Inisialisasi graph
        graph_3015 = new Graph_2511533015();
        visitedNodesSet_3015 = new HashSet<>();
        pathNodesSet_3015 = new HashSet<>();
        lastVisitedOrder_3015 = new ArrayList<>();
        lastExploredCount_3015 = 0;

        setTitle("Pencarian Jalur BFS dan DFS - NIM 2511533015");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(1000, 700);
        setLayout(new BorderLayout());

        // Panel atas untuk kontrol
        JPanel controlPanel_3015 = new JPanel(new FlowLayout());
```

```

controlPanel_3015.add(new JLabel("Lokasi Awal:"));
startCombo_3015 = new JComboBox<>(graph_3015.getNodes_3015().toArray(new String[0]));
controlPanel_3015.add(startCombo_3015);
controlPanel_3015.add(new JLabel("Lokasi Tujuan:"));
goalCombo_3015 = new JComboBox<>(graph_3015.getNodes_3015().toArray(new String[0]));
controlPanel_3015.add(goalCombo_3015);

JButton bfsButton_3015 = new JButton("BFS");
JButton dfsButton_3015 = new JButton("DFS");
JButton resetButton_3015 = new JButton("RESET");

// Memberi warna pada tombol sesuai permintaan
bfsButton_3015.setBackground(Color.GREEN);
bfsButton_3015.setOpaque(true);
dfsButton_3015.setBackground(Color.ORANGE);
dfsButton_3015.setOpaque(true);
resetButton_3015.setBackground(Color.RED);
resetButton_3015.setOpaque(true);
// Opsional: ubah warna teks agar kontras
bfsButton_3015.setForeground(Color.BLACK);
dfsButton_3015.setForeground(Color.BLACK);
resetButton_3015.setForeground(Color.WHITE);

controlPanel_3015.add(bfsButton_3015);
controlPanel_3015.add(dfsButton_3015);
controlPanel_3015.add(resetButton_3015);
add(controlPanel_3015, BorderLayout.NORTH);

// Area gambar graph
drawingPanel_3015 = new DrawingPanel_2511533015();
drawingPanel_3015.setBackground(Color.WHITE);
add(drawingPanel_3015, BorderLayout.CENTER);

// Area hasil pencarian
resultArea_3015 = new JTextArea(10, 40);
resultArea_3015.setEditable(false);
JScrollPane scrollPane_3015 = new JScrollPane(resultArea_3015);
add(scrollPane_3015, BorderLayout.SOUTH);

// Action listeners
bfsButton_3015.addActionListener(new ActionListener_3015() {

```

```

@Override
public void actionPerformed_3015(ActionEvent e) {
    performBFS_3015();
}
});

dfsButton_3015.addActionListener(new ActionListener_3015() {
    @Override
    public void actionPerformed_3015(ActionEvent e) {
        performDFS_3015();
    }
});

resetButton_3015.addActionListener(new ActionListener_3015() {
    @Override
    public void actionPerformed_3015(ActionEvent e) {
        resetGraph_3015();
    }
});

// Tampilkan graph awal
displayGraph_3015();
}

// Method BFS sesuai ketentuan
private void performBFS_3015() {
    String start = (String) startCombo_3015.getSelectedItem();
    String goal = (String) goalCombo_3015.getSelectedItem();
    if (start == null || goal == null) return;

    resetGraph_3015();

    SearchResult_3015 result = bfsSearch_3015(start, goal);
    if (result.path_3015.isEmpty()) {
        resultArea_3015.setText("Tidak ditemukan jalur dari " + start + " ke " + goal);
    } else {
        displayPath_3015(result.path_3015);
        visitedNodesSet_3015.addAll(result.visitedOrder_3015);
        pathNodesSet_3015.addAll(result.path_3015);
        lastVisitedOrder_3015 = result.visitedOrder_3015;
        lastExploredCount_3015 = result.exploredCount_3015;
    }
}

```



```

        resultArea_3015.append("\n\nJalur ditemukan menggunakan BFS\n");
        resultArea_3015.append("Jumlah node dieksplorasi: " + lastExploredCount_3015 + "\n");
        resultArea_3015.append("Urutan node dikunjungi: " + String.join(" -> ", lastVisitedOrder_3015) + "\n");
    }
    drawingPanel_3015.repaint();
}

```

// Method DFS sesuai ketentuan

```

private void performDFS_3015() {
    String start = (String) startCombo_3015.getSelectedItem();
    String goal = (String) goalCombo_3015.getSelectedItem();
    if (start == null || goal == null) return;

    resetGraph_3015();

    SearchResult_3015 result = dfsSearch_3015(start, goal);
    if (result.path_3015.isEmpty()) {
        resultArea_3015.setText("Tidak ditemukan jalur dari " + start + " ke " + goal);
    } else {
        displayPath_3015(result.path_3015);
        visitedNodesSet_3015.addAll(result.visitedOrder_3015);
        pathNodesSet_3015.addAll(result.path_3015);
        lastVisitedOrder_3015 = result.visitedOrder_3015;
        lastExploredCount_3015 = result.exploredCount_3015;

        resultArea_3015.append("\n\nJalur ditemukan menggunakan DFS\n");
        resultArea_3015.append("Jumlah node dieksplorasi: " + lastExploredCount_3015 + "\n");
        resultArea_3015.append("Urutan node dikunjungi: " + String.join(" -> ", lastVisitedOrder_3015) + "\n");
    }
    drawingPanel_3015.repaint();
}

```

// Implementasi BFS

```

private SearchResult_3015 bfsSearch_3015(String start, String goal) {
    Queue<String> queue_3015 = new LinkedList<>();
    Map<String, String> parent_3015 = new HashMap<>();
    Set<String> visited_3015 = new HashSet<>();
    List<String> order_3015 = new ArrayList<>();
    int explored_3015 = 0;
}

```

```

queue_3015.add(start);
visited_3015.add(start);
parent_3015.put(start, null);
order_3015.add(start);

while (!queue_3015.isEmpty()) {
    String current_3015 = queue_3015.poll();
    explored_3015++;

    if (current_3015.equals(goal)) {
        List<String> path_3015 = reconstructPath_3015(parent_3015, start, goal);
        return new SearchResult_3015(path_3015, order_3015, explored_3015);
    }

    for (String neighbor_3015 : graph_3015.getNeighbors_3015(current_3015)) {
        if (!visited_3015.contains(neighbor_3015)) {
            visited_3015.add(neighbor_3015);
            parent_3015.put(neighbor_3015, current_3015);
            queue_3015.add(neighbor_3015);
            order_3015.add(neighbor_3015);
        }
    }
}

return new SearchResult_3015(new ArrayList<>(), order_3015, explored_3015);
}

// Implementasi DFS (iteratif dengan stack)
private SearchResult_3015 dfsSearch_3015(String start, String goal) {
    Stack<String> stack_3015 = new Stack<>();
    Map<String, String> parent_3015 = new HashMap<>();
    Set<String> visited_3015 = new HashSet<>();
    List<String> order_3015 = new ArrayList<>();
    int explored_3015 = 0;

    stack_3015.push(start);
    visited_3015.add(start);
    parent_3015.put(start, null);
    order_3015.add(start);

    while (!stack_3015.isEmpty()) {
        String current_3015 = stack_3015.pop();

```

```

        explored_3015++;

        if (current_3015.equals(goal)) {
            List<String> path_3015 = reconstructPath_3015(parent_3015, start, goal);
            return new SearchResult_3015(path_3015, order_3015, explored_3015);
        }

        for (String neighbor_3015 : graph_3015.getNeighbors_3015(current_3015)) {
            if (!visited_3015.contains(neighbor_3015)) {
                visited_3015.add(neighbor_3015);
                parent_3015.put(neighbor_3015, current_3015);
                stack_3015.push(neighbor_3015);
                order_3015.add(neighbor_3015);
            }
        }
    }

    return new SearchResult_3015(new ArrayList<>(), order_3015, explored_3015);
}

// Membangun jalur dari parent map
private List<String> reconstructPath_3015(Map<String, String> parent_3015, String start, String goal) {
    List<String> path_3015 = new LinkedList<>();
    String current_3015 = goal;
    while (current_3015 != null) {
        path_3015.add(0, current_3015);
        current_3015 = parent_3015.get(current_3015);
    }
    if (path_3015.get(0).equals(start)) {
        return path_3015;
    }
    return new ArrayList<>();
}

// Menampilkan jalur di area hasil
public void displayPath_3015(List<String> path_3015) {
    if (path_3015.isEmpty()) {
        resultArea_3015.setText("Jalur: Tidak ada jalur\n");
    } else {
        resultArea_3015.setText("Jalur: " + String.join(" -> ", path_3015) + "\n");
        resultArea_3015.append("Node Dikunjungi: " + String.join(", ", lastVisitedOrder_3015) + "\n");
        resultArea_3015.append("Jumlah Node Dikunjungi: " + lastVisitedOrder_3015.size() + "\n");
    }
}

```

```

    }
}

// Menampilkan graph (cetak adjacency list ke console)
public void displayGraph_3015() {
    System.out.println("==== Representasi Graph (Adjacency List) =====");
    for (String node_3015 : graph_3015.getNodes_3015()) {
        System.out.print(node_3015 + " -> ");
        List<String> neighbors_3015 = graph_3015.getNeighbors_3015(node_3015);
        System.out.println(String.join(" ", neighbors_3015));
    }
    System.out.println("=====");
}

// Reset graph ke keadaan awal
public void resetGraph_3015() {
    visitedNodesSet_3015.clear();
    pathNodesSet_3015.clear();
    lastVisitedOrder_3015.clear();
    lastExploredCount_3015 = 0;
    resultArea_3015.setText("");
    drawingPanel_3015.repaint();
}

// Kelas untuk menyimpan hasil pencarian
private class SearchResult_3015 {
    List<String> path_3015;
    List<String> visitedOrder_3015;
    int exploredCount_3015;

    SearchResult_3015(List<String> path_3015, List<String> visitedOrder_3015, int exploredCount_3015) {
        this.path_3015 = path_3015;
        this.visitedOrder_3015 = visitedOrder_3015;
        this.exploredCount_3015 = exploredCount_3015;
    }
}

// Interface ActionListener dengan akhiran 3015
private interface ActionListener_3015 extends ActionListener {
    void actionPerformed_3015(ActionEvent e);
    default void actionPerformed(ActionEvent e) { actionPerformed_3015(e); }
}

```

```

    }

    // Panel untuk menggambar graph
    private class DrawingPanel_2511533015 extends JPanel {
        @Override
        protected void paintComponent(Graphics g_3015) {
            super.paintComponent(g_3015);
            Graphics2D g2d_3015 = (Graphics2D) g_3015;
            g2d_3015.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

            // Gambar edges
            g2d_3015.setColor(Color.BLACK);
            for (String[] edge_3015 : graph_3015.getEdges_3015()) {
                Point p1_3015 = graph_3015.getNodePosition_3015(edge_3015[0]);
                Point p2_3015 = graph_3015.getNodePosition_3015(edge_3015[1]);
                if (p1_3015 != null && p2_3015 != null) {
                    g2d_3015.drawLine(p1_3015.x, p1_3015.y, p2_3015.x, p2_3015.y);
                }
            }

            // Gambar node
            for (String node_3015 : graph_3015.getNodes_3015()) {
                Point pos_3015 = graph_3015.getNodePosition_3015(node_3015);
                if (pos_3015 == null) continue;

                // Warna node berdasarkan status
                if (pathNodesSet_3015.contains(node_3015)) {
                    g2d_3015.setColor(Color.GREEN);
                } else if (visitedNodesSet_3015.contains(node_3015)) {
                    g2d_3015.setColor(Color.CYAN);
                } else {
                    g2d_3015.setColor(Color.LIGHT_GRAY);
                }

                g2d_3015.fillOval(pos_3015.x - 20, pos_3015.y - 20, 40, 40);
                g2d_3015.setColor(Color.BLACK);
                g2d_3015.drawOval(pos_3015.x - 20, pos_3015.y - 20, 40, 40);
                FontMetrics fm_3015 = g2d_3015.getFontMetrics();
                int textWidth_3015 = fm_3015.stringWidth(node_3015);
                g2d_3015.drawString(node_3015, pos_3015.x - textWidth_3015/2, pos_3015.y + 5);
            }
        }
    }

```

```

    }
}

// Kelas graph dengan akhiran NIM 2511533015
private static class Graph_2511533015 {
    private Map<String, List<String>>> adjList_3015;
    private List<String> nodes_3015;
    private List<String[]> edges_3015;
    private Map<String, Point> positions_3015;

    public Graph_2511533015() {
        adjList_3015 = new HashMap<>();
        nodes_3015 = new ArrayList<>();
        edges_3015 = new ArrayList<>();
        positions_3015 = new HashMap<>();
        initGraph_3015();
    }

    private void initGraph_3015() {
        // Daftar node (minimal 10 node)
        String[] nodeNames_3015 = {
            "Rektorat", "FTI", "Perpus", "Masjid", "PKM",
            "Lab Jaringan", "Labor AI", "Gedung Serbaguna",
            "Koperasi", "Kantin Pusat", "Dosen", "Auditorium"
        };

        for (String node_3015 : nodeNames_3015) {
            nodes_3015.add(node_3015);
            adjList_3015.put(node_3015, new ArrayList<>());
        }

        // Posisi node (x,y) dalam panel
        positions_3015.put("Rektorat", new Point(200, 100));
        positions_3015.put("FTI", new Point(350, 100));
        positions_3015.put("Perpus", new Point(500, 150));
        positions_3015.put("Masjid", new Point(650, 100));
        positions_3015.put("PKM", new Point(300, 250));
        positions_3015.put("Lab Jaringan", new Point(450, 250));
        positions_3015.put("Labor AI", new Point(600, 250));
        positions_3015.put("Gedung Serbaguna", new Point(500, 380));
        positions_3015.put("Koperasi", new Point(200, 380));
        positions_3015.put("Kantin Pusat", new Point(350, 400));
    }
}

```

```

positions_3015.put("Dosen", new Point(700, 380));
positions_3015.put("Auditorium", new Point(800, 250));

// Daftar edge (minimal 15 edge)
String[][] edgeList_3015 = {
    {"Rektorat", "FTI"}, {"Rektorat", "PKM"}, {"Rektorat", "Perpus"},
    {"FTI", "Perpus"}, {"FTI", "Lab Jaringan"}, {"FTI", "Labor AI"},
    {"Perpus", "Masjid"}, {"PKM", "Gedung Serbaguna"}, {"PKM", "Kantin Pusat"},
    {"PKM", "Koperasi"}, {"Masjid", "Gedung Serbaguna"},
    {"Lab Jaringan", "Labor AI"}, {"Labor AI", "Auditorium"},
    {"Auditorium", "Dosen"}, {"Dosen", "Gedung Serbaguna"},
    {"Koperasi", "Kantin Pusat"}
};

for (String[] edge_3015 : edgeList_3015) {
    addEdge_3015(edge_3015[0], edge_3015[1]);
    edges_3015.add(new String[]{edge_3015[0], edge_3015[1]});
}

private void addEdge_3015(String u, String v) {
    adjList_3015.get(u).add(v);
    adjList_3015.get(v).add(u);
}

public List<String> getNodes_3015() { return nodes_3015; }
public List<String> getNeighbors_3015(String node) { return adjList_3015.get(node); }
public List<String[]> getEdges_3015() { return edges_3015; }
public Point getNodePosition_3015(String node) { return positions_3015.get(node); }

public static void main(String[] args_3015) {
    SwingUtilities.invokeLater(() -> {
        PencarianJalur_2511533015 frame_3015 = new PencarianJalur_2511533015();
        frame_3015.setVisible(true);
    });
}
}

```

## 2. Penjelasan

### 1. Import library

```
import javax.swing.*;  
import java.awt.*;  
import java.awt.event.ActionEvent;  
import java.awt.event.ActionListener;  
import java.util.*;  
import java.util.List;
```

fungsi:

digunakan untuk mengimpor berbagai library

library yang digunakan:

javax.swing: membuat tampilan GUI

java.awt: mengatur layout dan gambar

java.awt.event: menangani event tombol

java.util: struktur data seperti queue, stack, map, list

### 2. Deklarasi class

```
public class PencarianJalur_2511533015 extends JFrame
```

fungsi:

class utama program

karena mewarisi (extends) JFrame, maka class ini menjadi sebuah jendela aplikasi

### 3. Deklarasi variabel

```
private Graph_2511533015 graph_3015;  
private JComboBox<String> startCombo_3015;  
private JComboBox<String> goalCombo_3015;  
private JTextArea resultArea_3015;  
private DrawingPanel_2511533015 drawingPanel_3015;
```



fungsi masing masing:

`^ graph_3015`

Menyimpan struktur graf

`^ startCombo_3015`

Dropdown untuk memilih Lokasi awal

Contoh:

Rektorat

FTI

Perpus

Masjid

`^ goalCombo_3015`

Dropdown untuk memilih tujuan

`^ resultArea_3015`

Menampilkan hasil pencarian

Misalnya:

Jalur:

Rektorat -> FTI -> Labor AI

Jumlah Node:

5

`^ drawingPanel_3015`

Digunakan untuk menggambar graf

#### 4. Constructor

```
public PencarianJalur_2511533015()
```

constructor dijalankan Ketika objek dibuat

^ membuat graph

```
graph_3015 = new Graph_2511533015();
```

membuat graf kampus

^mengatur judul

```
setTitle("Pencarian Jalur BFS dan DFS");
```

memberikan judul pada aplikasi

^mengatur ukuran

```
setSize(1000,700);
```

ukuran jendela:

Lebar = 1000

Tinggi = 700

#### 5. Membuat panel control

```
JPanel controlPanel_3015 = new JPanel(new FlowLayout());
```

Panel ini berisi:

# Lokasi awal

# Lokasi tujuan

# tombol BFS

# tombol DFS

# Tombol reset

#### 6. Membuat comboBox

```
startCombo_3015 = new JComboBox<>  
goalCombo_3015 = new JComboBox<>
```

digunakan untuk memilih node awal dan tujuan.

Misalnya:

Start:

Rektorat

Goal:

Auditorium

#### 7. Membuat tombol

```
JButton bfsButton_3015  
JButton dfsButton_3015  
JButton resetButton_3015
```

Fungsi:

^ Tombol BFS

Menjalankan algoritma BFS

Warna: hijau

^ Tombol DFS

Menjalankan algoritma DFS

Warna: orange

^ Tombol RESET

Menghapus hasil pencarian

Warna: merah

## 8. Event tombol

^ Tombol BFS

```
bfsButton_3015.addActionListener(...)
```

saat ditekan:

```
performBFS_3015();
```

^ tombol DFS

Saat ditekan:

```
performDFS_3015();
```

^ Tombol reset

Saat ditekan:

```
resetGraph_3015();
```

## 9. Method BFS

```
private void performBFS_3015()
```

Langkah kerja

1

Mengambil node awal dan tujuan

String start

String goal

2

Reset tampilan

```
resetGraph_3015();
```

3

Menjalankan BFS

```
SearchResult_3015 result =  
bfsSearch_3015(start, goal);
```

4

Jika jalur ditemukan

Maka program akan:

# menampilkan jalur

# menampilkan jumlah node

# menampilkan urutan kunjungan

Contoh:

Jalur:

Rektorat -> FTI -> Labor AI

Jumlah Node:

4

Urutan:

Rektorat

FTI

Perpus

Labor AI

## 10. Algoritma BFS

```
private SearchResult_3015 bfsSearch_3015()
```

Menggunakan:

^ queue:

Queue<String> queue

# Prinsip:

FIFO

(First In First Out)

# Misalnya graf:

```
  A
 /\
B  C
 |
D
```

# Proses:

Queue=[A]

A keluar

Masuk B,C

Queue=[B,C]

B keluar

Masuk D

Queue=[C,D]

C keluar

Queue=[D]

D keluar

# Output:

A B C D

## 11. Method DFS

```
private void performDFS_3015()
```

fungsinya hamper sama dengan BFS

bedanya menggunakan:

```
dfsSearch_3015()
```

## 12. Algoritma DFS

```
private SearchResult_3015 dfsSearch_3015()
```

# menggunakan:

```
Stack<String>
```

# Prinsip:

LIFO

(Last In First Out)

# Contoh:

A

/\

B C

|

D

# Proses:

A

↓

B

↓

D

↑

B

↓

C

# output

A B D C

### 13. Method reconstructPath()

```
private List<String> reconstructPath_3015()
```

method ini membangun Kembali jalur yang telah ditemukan

misalnya parent map:

A = null

B = A

C = B

D = C

Maka jalur:



A -> B -> C -> D

#### 14. Method displayPath()

displayPath\_3015()

digunakan untuk menampilkan hasil ke TextArea

contoh:

Jalur:

Rektorat -> PKM -> Gedung Serbaguna

Node Dikunjungi:

Rektorat,

PKM,

Gedung Serbaguna

Jumlah:

3

#### 15. Method resetGraph()

resetGraph\_3015()

fungsinya:

# menghapus node yang dikunjungi

# menghapus jalur

# menghapus hasil pencarian

# menggambar ulang graf

#### 16. Class searchResult\_3015

private class SearchResult\_3015

digunakan untuk menyimpan hasil pencarian

berisi:

List path

List visitedOrder

int exploredCount

contoh:

Jalur:

A-B-C

Visited:

A,B,C,D

Explored:

4

#### 17. Interface ActionListener\_3015

```
private interface ActionListener_3015
```

digunakan agar penanganan event tombol lebih rapi

misalnya:

Klik BFS

↓

actionPerformed\_3015()

↓

performBFS()

#### 18. Class DrawingPanel\_2511533015

Class ini bertugas menggambar graf

^ menggambar edge

drawLine()

misalnya:

A ----- B

^ menggambar node

fillOval()

node Digambar sebagai lingkaran

^ pewarnaan node

a. Abu abu

node belum dikunjungi

b. biru muda (Cyan)

node sudah dikunjungi

c. hijau

node merupakan jalur hasil pencarian

misalnya:

A (Hijau)

|

B (Hijau)

|

C (Hijau)

D (Biru)

#### 19. Class graph\_2511533015

Class ini menyimpan seluruh graf

Struktur data

```
# Map<String,List<String>>
```

Contoh:

Rektorat -> FTI, PKM, Perpus

FTI -> Rektorat, Perpus

Perpus -> Rektorat, Masjid

- Node yang dibuat:

  1. Rektorat
  2. FTI
  3. Perpus
  4. Masjid
  5. PKM
  6. Lab jaringan
  7. Labor AI
  8. Gedung serbaguna
  9. Koperasi
  10. Kantin pusat
  11. Dosen

## 12. Auditorium

### 20. Edge yang dibuat

Contoh:

Rektorat --- FTI

Rektorat --- PKM

FTI --- Lab Jaringan

Labor AI --- Auditorium

Auditorium --- Dosen

Total terdapat lebih dari 15 edge

### 21. Method main ()

```
public static void main()
```

merupakan titik awal program

```
SwingUtilities.invokeLater(() -> {  
  
    PencarianJalur_2511533015 frame =  
    new PencarianJalur_2511533015();  
  
    frame.setVisible(true);  
  
});
```

Artinya:

# membuat jendela aplikasi

# menampilkan GUI

# pengguna dapat memilih Lokasi

# menekan BFS dan Dfs

# program mencari dan menampilkan jalur

## 22. Alur kerja program

Program dijalankan



Membuat Graph Kampus



Menampilkan GUI



Pilih Lokasi Awal dan Tujuan



Tekan Tombol

BFS atau DFS



Menjalankan Algoritma



Mencari Jalur



Menampilkan:

- Jalur
- Node yang dikunjungi
- Jumlah eksplorasi
- Warna pada graf

- Kesimpulan\

Program pencarianJalur\_2511533015 merupakan aplikasi simulasi pencarian jalur pada graf menggunakan algoritma BFS dan DFS dengan antarmuka grafis java Swing

Fitur utama program ini Adalah:

1. Membuat graf berbentuk adjacency list
2. Menampilkan graf secara visual
3. Melakukan pencarian jalur menggunakan BFS
4. Melakukan pencarian jalur menggunakan DFS
5. Menampilkan jalur hasil pencarian
6. Menampilkan jumlah node yang dieksplorasi
7. Menampilkan urutan node yang dikunjungi
8. Memberikan visualisasi warna pada node yang dikunjungi dan jalur yang ditemukan.\

### 3. Output

```
Problems Javadoc Declaration Search Console Git Staging
PencarianJalur_2511533015 [Java Application] C:\Users\user\p2\pool\plugins\org.eclipse.justj.openjdk.hot
===== Representasi Graph (Adjacency List) =====
Rektorat -> FTI, PKM, Perpus
FTI -> Rektorat, Perpus, Lab Jaringan, Labor AI
Perpus -> Rektorat, FTI, Masjid
Masjid -> Perpus, Gedung Serbaguna
PKM -> Rektorat, Gedung Serbaguna, Kantin Pusat, Koperasi
Lab Jaringan -> FTI, Labor AI
Labor AI -> FTI, Lab Jaringan, Auditorium
Gedung Serbaguna -> PKM, Masjid, Dosen
Koperasi -> PKM, Kantin Pusat
Kantin Pusat -> PKM, Koperasi
Dosen -> Auditorium, Gedung Serbaguna
Auditorium -> Labor AI, Dosen
=====
rawing
_3015
015;
_3015
015();
Set<>
```

